

# Project / File Structure

## 1. Document Information

| Item               | Details                                                                                                                    |
|--------------------|----------------------------------------------------------------------------------------------------------------------------|
| Project Name       | Elevator Gate                                                                                                              |
| Project Type       | Corporate Services Website                                                                                                 |
| Backend Framework  | Django                                                                                                                     |
| Database           | PostgreSQL                                                                                                                 |
| Frontend           | Django Templates, HTML, CSS, JavaScript                                                                                    |
| Architecture Style | Modular Django Applications                                                                                                |
| Document Status    | Sprint 2 – System Design                                                                                                   |
| Document Purpose   | Define the planned project folders, Django applications, configuration files, templates, static files, and media structure |

## 2. Purpose

This document defines the proposed project and file structure for the Elevator Gate website.

The purpose of the structure is to:

- Keep the project organized and easy to understand.
- Separate business domains into independent Django applications.
- Avoid placing all functionality inside one large application.
- Make the project easier to develop, test, maintain, and expand.
- Separate configuration, templates, static assets, uploaded files, and application logic.
- Support future deployment and environment configuration.
- Follow professional Django project organization practices.

## 3. High-Level Project Structure

```
elevator_gate/  
|
```

```
|— config/
|
|— apps/
|
|— templates/
|
|— static/
|
|— media/
|
|— locale/
|
|— docs/
|
|— tests/
|
|— requirements/
|
|— manage.py
|— requirements.txt
|— .env
|— .env.example
|— .gitignore
|— README.md
|— LICENSE
```

## 4. Complete Proposed File Structure

```
elevator_gate/
|
|— config/
|   |— __init__.py
|   |— asgi.py
|   |— urls.py
|   |— wsgi.py
|   |
|   └— settings/
|       |— __init__.py
|       |— base.py
|       |— development.py
|       └— production.py
|
|— apps/
```

```

├── __init__.py
├── core/
│   ├── migrations/
│   │   └── __init__.py
│   ├── templatetags/
│   │   ├── __init__.py
│   │   └── core_tags.py
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── context_processors.py
│   ├── forms.py
│   ├── models.py
│   ├── selectors.py
│   ├── services.py
│   ├── tests.py
│   ├── urls.py
│   └── views.py
├── pages/
│   ├── migrations/
│   │   └── __init__.py
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── selectors.py
│   ├── services.py
│   ├── tests.py
│   ├── urls.py
│   └── views.py
├── services/
│   ├── migrations/
│   │   └── __init__.py
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── forms.py
│   ├── models.py
│   ├── selectors.py
│   ├── services.py
│   ├── tests.py
│   ├── urls.py
│   └── views.py

```

```
|
|
|   └─ projects/
|       └─ migrations/
|           └─ __init__.py
|       └─ __init__.py
|       └─ admin.py
|       └─ apps.py
|       └─ filters.py
|       └─ models.py
|       └─ selectors.py
|       └─ services.py
|       └─ tests.py
|       └─ urls.py
|       └─ views.py
|
|   └─ blog/
|       └─ migrations/
|           └─ __init__.py
|       └─ __init__.py
|       └─ admin.py
|       └─ apps.py
|       └─ models.py
|       └─ selectors.py
|       └─ services.py
|       └─ tests.py
|       └─ urls.py
|       └─ views.py
|
|   └─ contact/
|       └─ migrations/
|           └─ __init__.py
|       └─ __init__.py
|       └─ admin.py
|       └─ apps.py
|       └─ forms.py
|       └─ models.py
|       └─ services.py
|       └─ tests.py
|       └─ urls.py
|       └─ views.py
|
|   └─ templates/
|       └─ base.html
|       └─ 404.html
|       └─ 500.html
|
|
```



```
|
|
|
|   └─ global.css
|
|   └─ components/
|       ├── buttons.css
|       ├── cards.css
|       ├── forms.css
|       ├── navbar.css
|       ├── footer.css
|       ├── pagination.css
|       └─ faq.css
|
|   └─ pages/
|       ├── home.css
|       ├── about.css
|       ├── services.css
|       ├── projects.css
|       ├── blog.css
|       └─ contact.css
|
|   └─ utilities.css
|   └─ main.css
|
| └─ js/
|     ├── main.js
|     ├── navigation.js
|     ├── forms.js
|     ├── faq.js
|     ├── filters.js
|     └─ animations.js
|
| └─ images/
|     ├── branding/
|         ├── logo.svg
|         ├── logo-light.svg
|         └─ favicon.ico
|
|     ├── hero/
|     ├── services/
|     ├── projects/
|     ├── blog/
|     ├── icons/
|     └─ placeholders/
|
| └─ fonts/
| └─ vendor/
```

```
├─ media/
│   ├── services/
│   ├── projects/
│   ├── blog/
│   ├── testimonials/
│   └─ site/
├─ locale/
│   ├── ar/
│   │   └─ LC_MESSAGES/
│   └─ en/
│       └─ LC_MESSAGES/
├─ docs/
│   ├── 01_Project_Brief.md
│   ├── 02_Functional_Requirements.md
│   ├── 03_Sitemap.md
│   ├── 04_User_Flow.md
│   ├── 05_Design_System.md
│   ├── 06_Wireframes.md
│   ├── 07_High_Fidelity_Design.md
│   ├── 08_System_Architecture.md
│   ├── 09_Database_Design.md
│   └─ 10_Project_File_Structure.md
├─ tests/
│   ├── __init__.py
│   ├── test_homepage.py
│   ├── test_navigation.py
│   ├── test_services.py
│   ├── test_projects.py
│   ├── test_blog.py
│   └─ test_contact.py
├─ requirements/
│   ├── base.txt
│   ├── development.txt
│   └─ production.txt
├─ manage.py
├─ requirements.txt
├─ .env
├─ .env.example
├─ .gitignore
├─ README.md
└─ LICENSE
```

---

## 5. Main Directory Responsibilities

### 5.1 `config/`

The `config` directory contains the central Django project configuration.

It does not contain the website's business content. Instead, it connects and configures all Django applications.

#### Main files

```
config/  
├─ asgi.py  
├─ urls.py  
├─ wsgi.py  
└─ settings/
```

#### Responsibilities

- Django project settings.
- Root URL configuration.
- ASGI and WSGI configuration.
- Database connection settings.
- Static and media configuration.
- Security settings.
- Installed applications.
- Middleware.
- Template configuration.
- Internationalization settings.

---

### 5.2 `config/settings/`

The Django settings are divided by environment.

```
settings/  
├─ base.py  
├─ development.py  
└─ production.py
```



### `base.py`

Contains settings shared by all environments.

Examples:

- Installed applications.
- Middleware.
- Template settings.
- Authentication settings.
- Language and timezone.
- Static and media paths.
- Common security configuration.

### `development.py`

Contains settings used during local development.

Examples:

```
DEBUG = True
```

It may also contain:

- Local development database settings.
- Django Debug Toolbar configuration.
- Development email backend.
- Less restrictive local security settings.

### `production.py`

Contains settings used after deployment.

Examples:

```
DEBUG = False
```

It should include:

- Production database configuration.
- Allowed hosts.
- Secure cookies.
- HTTPS settings.
- Production email configuration.
- Logging.

- Security headers.
- 

## 6. Django Applications

The project will be divided into the following Django applications:

```
core
pages
services
projects
blog
contact
```

Each application owns one specific area of the system.

---

### 6.1 `core` Application

The `core` application contains shared site-wide functionality.

#### Responsibilities

- Global site settings.
- Header and footer information.
- Social media links.
- Shared SEO settings.
- Testimonials.
- Reusable utilities.
- Shared context processors.
- Shared template tags.

#### Possible models

```
SiteSettings
SocialLink
Testimonial
SEOSettings
```

#### Important rule

The `core` application should not contain content that clearly belongs to another domain.

For example:

- Services belong to `services`.
  - Projects belong to `projects`.
  - Articles belong to `blog`.
  - Contact messages belong to `contact`.
- 

## 6.2 `pages` Application

The `pages` application manages general informational pages that do not require a dedicated business application.

### Responsibilities

- Homepage.
- About page.
- FAQ page.
- General static page content.
- Homepage section assembly.

### Possible models

```
HomePageContent  
AboutPageContent  
FAQ
```

The homepage may display data from several applications, including:

- Featured services.
- Featured projects.
- Latest blog posts.
- Testimonials.
- FAQs.

The `pages` application may retrieve this information through selectors or service functions without owning the original data.

---

## 6.3 `services` Application

The `services` application manages elevator services provided by the company.

## Responsibilities

- Services listing page.
- Service details pages.
- Service features and benefits.
- Featured services.
- Service SEO data.
- Service ordering.

## Possible models

```
Service
ServiceFeature
```

## Main URLs

```
/services/
/services/<slug>/
```

The service detail page should use the unique service `slug`, not the database ID.

Example:

```
/services/elevator-maintenance/
```

Instead of:

```
/services/4/
```

---

## 6.4 `projects` Application

The `projects` application manages completed elevator projects.

## Responsibilities

- Projects listing.
- Project details.
- Project categories.
- Project filtering.
- Project gallery.
- Featured projects.

## Possible models

```
Project
ProjectCategory
ProjectImage
```

## Main URLs

```
/projects/
/projects/<slug>/
/projects/category/<slug>/
```

---

## 6.5 `blog` Application

The `blog` application manages articles and SEO content.

### Responsibilities

- Blog listing.
- Blog article details.
- Blog categories.
- Publishing status.
- Article metadata.
- SEO content.
- Related posts.

## Possible models

```
BlogPost
BlogCategory
```

## Main URLs

```
/blog/
/blog/<slug>/
/blog/category/<slug>/
```

---

## 6.6 `contact` Application

The `contact` application manages contact forms and customer messages.

### Responsibilities

- Contact page.
- Contact form validation.
- Saving contact messages.
- Sending email notifications.
- Displaying success or error messages.
- Managing customer enquiries from Django Admin.

### Main model

ContactMessage

### Main URLs

`/contact/`  
`/contact/success/`

---

## 7. Standard Application Files

Each Django application may contain the following files:

```
app_name/  
├─ admin.py  
├─ apps.py  
├─ forms.py  
├─ models.py  
├─ selectors.py  
├─ services.py  
├─ tests.py  
├─ urls.py  
└─ views.py
```

## 7.1 models.py

Contains database models owned by the application.

Example:

```
class Service(models.Model):
    title = models.CharField(max_length=200)
    slug = models.SlugField(unique=True)
```

---

## 7.2 views.py

Receives HTTP requests and returns HTTP responses.

Views should remain relatively small and should not contain all business logic or complex database queries.

A view may:

1. Receive the request.
  2. Call a selector or service function.
  3. Prepare the template context.
  4. Return the rendered response.
- 

## 7.3 urls.py

Defines the URL routes belonging to the application.

Example:

```
urlpatterns = [
    path("", views.service_list, name="service_list"),
    path("<slug:slug>/", views.service_detail, name="service_detail"),
]
```

---

## 7.4 forms.py

Contains Django forms and validation rules.

Examples:

- Contact form.
  - Search form.
  - Project filtering form.
  - Admin-related custom forms.
- 

## 7.5 `admin.py`

Registers models in the Django Admin panel.

It may define:

- List columns.
  - Filters.
  - Search fields.
  - Slug auto-generation.
  - Field groups.
  - Inline related models.
- 

## 7.6 `selectors.py`

Contains reusable database read operations.

Selectors are used for querying and retrieving data.

Examples:

```
def get_featured_services():
    return Service.objects.filter(
        is_active=True,
        is_featured=True
    ).order_by("display_order")
```

Another example:

```
def get_published_blog_posts():
    return BlogPost.objects.filter(
        status=BlogPost.Status.PUBLISHED
    )
```



Selectors help prevent complex queries from being repeated inside multiple views.

---

## 7.7 `services.py`

Contains business operations and actions that may modify data or coordinate several steps.

Examples:

- Saving a contact message.
- Sending a contact notification email.
- Publishing a blog post.
- Generating a unique slug.
- Processing an enquiry.
- Updating related project data.

Example:

```
def create_contact_message(*, cleaned_data):
    message = ContactMessage.objects.create(**cleaned_data)
    send_contact_notification(message)
    return message
```

---

## 7.8 `tests.py`

Contains automated tests related to the application.

Tests may verify:

- Models.
- URLs.
- Views.
- Forms.
- Selectors.
- Services.
- Access permissions.
- Expected HTTP status codes.

When the application becomes larger, `tests.py` may be replaced with a tests directory:

```
tests/
├─ __init__.py
├─ test_models.py
```

```
|— test_views.py
|— test_forms.py
|— test_selectors.py
└— test_services.py
```

---

## 8. Templates Structure

Templates are stored in one central project-level `templates` directory.

```
templates/
|— base.html
|— includes/
|— components/
|— pages/
|— services/
|— projects/
|— blog/
└— contact/
```

---

### 8.1 `base.html`

The main layout inherited by other pages.

It should contain:

- HTML document structure.
- Metadata.
- Main CSS imports.
- Header.
- Navigation.
- Main content block.
- Footer.
- JavaScript imports.

Example:

```
<!DOCTYPE html>
<html lang="ar" dir="rtl">
<head>
    {% include "includes/seo_meta.html" %}
```

```
</head>
<body>

    {% include "includes/header.html" %}

    <main>
        {% block content %}
        {% endblock %}
    </main>

    {% include "includes/footer.html" %}

</body>
</html>
```

---

## 8.2 includes/

Contains repeated sections that appear on many pages.

Examples:

```
header.html
navbar.html
footer.html
messages.html
breadcrumbs.html
pagination.html
seo_meta.html
```

These files represent page-level shared sections.

---

## 8.3 components/

Contains smaller reusable interface components.

Examples:

```
service_card.html
project_card.html
blog_card.html
testimonial_card.html
```

```
faq_item.html
button.html
```

A component receives data and renders a repeated design element.

Example:

```
{% include "components/service_card.html" with service=service %}
```

---

## 8.4 Application Template Directories

Templates are grouped according to their Django application.

Example:

```
templates/services/
├─ service_list.html
└─ service_detail.html
```

This naming structure prevents template name conflicts and makes files easier to locate.

---

# 9. Static Files Structure

Static files are files owned by the website source code and do not change through Django Admin.

```
static/
├─ css/
├─ js/
├─ images/
├─ fonts/
└─ vendor/
```

---

## 9.1 CSS

CSS is divided into:

```
css/  
├─ base/  
├─ components/  
├─ pages/  
├─ utilities.css  
└─ main.css
```

### base/

Contains global styling foundations:

- CSS reset.
- Design tokens.
- Typography.
- Global element styling.

### components/

Contains component-specific styling:

- Buttons.
- Cards.
- Forms.
- Navigation.
- Footer.
- FAQ accordion.

### pages/

Contains page-specific styles:

- Homepage.
- About page.
- Services.
- Projects.
- Blog.
- Contact.

### variables.css

Contains design-system values.

Example:

```
:root {  
  --color-primary: #1f3c5b;
```

```
--color-secondary: #c8a45d;
--color-background: #f7f8fa;

--spacing-sm: 0.5rem;
--spacing-md: 1rem;
--spacing-lg: 2rem;

--radius-sm: 4px;
--radius-md: 8px;
--radius-lg: 16px;
}
```

---

## 9.2 JavaScript

JavaScript files are separated by responsibility.

```
js/
├─ main.js
├─ navigation.js
├─ forms.js
├─ faq.js
├─ filters.js
└─ animations.js
```

Examples:

- Mobile navigation.
- FAQ accordion.
- Project filtering.
- Form interactions.
- Scroll animations.

---

## 9.3 Images

```
images/
├─ branding/
├─ hero/
├─ services/
├─ projects/
└─ blog/
```

```
|— icons/  
|— placeholders/
```

These images are included directly in the website source code.

Examples:

- Logo.
- Default hero images.
- Interface icons.
- Placeholder images.
- Decorative backgrounds.

---

## 10. Static Files vs Media Files

Static and media files must not be confused.

### Static files

Static files are part of the website design and code.

Examples:

- Logo.
- CSS files.
- JavaScript files.
- Default icons.
- Decorative images.
- Local fonts.

Location:

```
static/
```

### Media files

Media files are uploaded dynamically through Django Admin or forms.

Examples:

- Service images.
- Project gallery images.

- Blog cover images.
- Testimonial images.
- Uploaded site settings images.

Location:

```
media/
```

The database stores the path to the file, while the actual file is stored inside the media directory or a cloud storage service.

---

## 11. Media Directory Structure

```
media/  
├─ services/  
├─ projects/  
├─ blog/  
├─ testimonials/  
└─ site/
```

Example model field:

```
image = models.ImageField(  
    upload_to="services/"  
)
```

A service image would then be stored under:

```
media/services/
```

---

## 12. Requirements Structure

Python dependencies may initially be listed inside:

```
requirements.txt
```

As the project grows, dependencies should be divided by environment:



```
requirements/  
├─ base.txt  
├─ development.txt  
└─ production.txt
```

### base.txt

Shared packages:

```
Django  
psycopg  
Pillow  
python-decouple
```

### development.txt

Development-only tools:

```
-r base.txt  
django-debug-toolbar  
pytest  
pytest-django  
coverage
```

### production.txt

Production packages:

```
-r base.txt  
unicorn  
whitenoise
```

---

## 13. Environment Variables

Sensitive configuration values should be stored in a `.env` file.

Example:

```
SECRET_KEY=replace-with-secret-key
DEBUG=True
DATABASE_NAME=elevator_gate
DATABASE_USER=postgres
DATABASE_PASSWORD=replace-with-password
DATABASE_HOST=localhost
DATABASE_PORT=5432
EMAIL_HOST_USER=example@example.com
EMAIL_HOST_PASSWORD=replace-with-password
```

The `.env` file must not be uploaded to GitHub.

---

## 13.1 `.env.example`

The `.env.example` file contains the required variable names without real secret values.

Example:

```
SECRET_KEY=
DEBUG=
DATABASE_NAME=
DATABASE_USER=
DATABASE_PASSWORD=
DATABASE_HOST=
DATABASE_PORT=
EMAIL_HOST_USER=
EMAIL_HOST_PASSWORD=
```

This allows another developer to understand which environment variables are required.

---

## 14. `.gitignore`

The `.gitignore` file prevents unnecessary or sensitive files from being committed.

Proposed content:

```
# Python
__pycache__/
*.py[cod]
*.pyo
```

```
# Virtual environments
.venv/
venv/
env/

# Environment variables
.env

# Django
db.sqlite3
media/
staticfiles/

# Test and coverage files
.pytest_cache/
.coverage
htmlcov/

# IDE
.vscode/
.idea/

# macOS
.DS_Store

# Logs
*.log
logs/
```

The `media/` directory may be excluded because uploaded files normally belong to the deployment environment rather than the source repository.

---

## 15. Documentation Directory

Project documentation will be stored inside:

```
docs/
```

Proposed documents:

```
01_Project_Brief.md
02_Functional_Requirements.md
```

```
03_Sitemap.md
04_User_Flow.md
05_Design_System.md
06_Wireframes.md
07_High_Fidelity_Design.md
08_System_Architecture.md
09_Database_Design.md
10_Project_File_Structure.md
```

Future documents may include:

```
11_API_Documentation.md
12_Test_Plan.md
13_Deployment_Guide.md
14_User_Manual.md
15_Project_Case_Study.md
```

---

## 16. Testing Structure

During early development, each application may use its default `tests.py` file.

As testing expands, tests should be organized by application or test category.

Example:

```
apps/services/tests/
├── __init__.py
├── test_models.py
├── test_views.py
├── test_urls.py
├── test_selectors.py
└── test_admin.py
```

Project-level integration tests may remain inside:

```
tests/
```

Examples:

- Homepage integration.

- Main navigation.
  - Contact form workflow.
  - Project filtering.
  - Blog pagination.
  - Error pages.
- 

## 17. URL Structure

The root URL configuration will include each application's URLs.

Example:

```
urlpatterns = [  
    path("admin/", admin.site.urls),  
    path("", include("apps.pages.urls")),  
    path("services/", include("apps.services.urls")),  
    path("projects/", include("apps.projects.urls")),  
    path("blog/", include("apps.blog.urls")),  
    path("contact/", include("apps.contact.urls")),  
]
```

Each application is responsible for its own internal URLs.

---

## 18. URL Naming Convention

URL names should use clear names with application namespaces.

Example:

```
app_name = "services"  
  
urlpatterns = [  
    path("", views.service_list, name="list"),  
    path("<slug:slug>/", views.service_detail, name="detail"),  
]
```

Template usage:

```
{% url "services:list" %}
```

```
{% url "services:detail" service.slug %}
```

This avoids URL name conflicts between different applications.

---

## 19. Python Naming Conventions

The project should follow standard Python naming conventions.

### Files and modules

Use lowercase snake case:

```
context_processors.py  
email_services.py  
test_models.py
```

### Functions and variables

Use lowercase snake case:

```
get_featured_services()  
contact_message  
published_posts
```

### Classes and models

Use PascalCase:

```
Service  
ProjectCategory  
ContactMessage
```

### Constants

Use uppercase snake case:

```
MAX_UPLOAD_SIZE  
DEFAULT_PAGE_SIZE
```

---

## 20. Template Naming Conventions

Template filenames should be descriptive and lowercase.

Recommended:

```
service_list.html  
service_detail.html  
project_card.html  
contact_success.html
```

Avoid vague names such as:

```
page1.html  
new.html  
test.html  
data.html
```

---

## 21. Static File Naming Conventions

Static filenames should use lowercase names separated by hyphens or underscores consistently.

Examples:

```
hero-elevator.jpg  
service-maintenance.jpg  
logo-light.svg  
project-placeholder.webp
```

Do not use:

Final Logo New 2.png  
image copy latest.jpg  
Untitled-1.png

## 22. File Responsibility Rules

To keep the project maintainable, the following rules should be applied:

1. Models contain database structure and model-level behavior.
2. Views handle HTTP request and response flow.
3. Forms handle input validation.
4. Selectors contain reusable read queries.
5. Services contain business operations.
6. Templates contain presentation markup.
7. Static files contain design assets and frontend behavior.
8. Uploaded content belongs to media storage.
9. Secrets belong to environment variables.
10. Each Django application owns only its business domain.

## 23. Example Request Flow

When a visitor opens a service detail page:

```
Browser Request
  ↓
config/urls.py
  ↓
apps/services/urls.py
  ↓
apps/services/views.py
  ↓
apps/services/selectors.py
  ↓
Service Model
  ↓
PostgreSQL Database
  ↓
templates/services/service_detail.html
```



↓  
HTML Response

Example URL:

/services/elevator-maintenance/

Possible view:

```
def service_detail(request, slug):
    service = get_service_by_slug(slug=slug)

    return render(
        request,
        "services/service_detail.html",
        {"service": service},
    )
```

Possible selector:

```
def get_service_by_slug(*, slug):
    return get_object_or_404(
        Service,
        slug=slug,
        is_active=True,
    )
```

---

## 24. Example Contact Form Flow

```
User submits contact form
↓
contact/views.py
↓
contact/forms.py validates data
↓
contact/services.py processes submission
↓
ContactMessage saved in database
↓
Notification email is sent
```

↓  
Success page is displayed

This structure prevents the view from containing all validation, database, and email logic.

---

## 25. Initial Implementation Scope

The full proposed structure represents the planned target architecture.

Not every file must be created on the first development day.

The initial implementation may begin with:

```
config/  
apps/  
  core/  
  pages/  
  services/  
  projects/  
  blog/  
  contact/  
templates/  
static/  
media/  
manage.py  
requirements.txt  
.env  
.env.example  
.gitignore  
README.md
```

Additional files such as:

```
selectors.py  
services.py  
filters.py  
templatetags/  
separate settings files  
separate test directories
```

should be introduced when the related functionality is required.

This avoids creating empty files without a clear purpose while preserving the planned architecture.

---

## 26. Architecture Decision Summary

The Elevator Gate project will use:

- One Django project configuration directory named `config`.
  - A central `apps` package for local Django applications.
  - Separate applications for core, pages, services, projects, blog, and contact.
  - Central project-level templates.
  - Separate static and uploaded media directories.
  - Environment variables for secrets and deployment configuration.
  - PostgreSQL as the primary database.
  - Slug-based public detail URLs.
  - Selectors for reusable database reads.
  - Services for business operations.
  - Environment-specific settings for development and production.
  - Organized project documentation under `docs`.
- 

## 27. Final Approved Structure

The approved primary Django applications are:

```
core
pages
services
projects
blog
contact
```

The approved primary project directories are:

```
config
apps
templates
static
media
locale
docs
```

```
tests
requirements
```

This structure will act as the technical reference when the Django project is initialized during the development sprint.

---

## 28. Deliverable

The deliverable for this task is:

```
10_Project_File_Structure.md
```

The document includes:

- Complete proposed folder tree.
- Django application responsibilities.
- Configuration structure.
- Template structure.
- Static and media organization.
- Environment variables.
- Dependency management.
- Testing organization.
- Naming conventions.
- Request-flow examples.
- Initial implementation scope.